allocating another one. Then at line 2, the block of size 5 can be allocated at `0x200` or `0x205` in the former case and at `0x100` or `0x105` in the latter case because there is enough space left. Among those twin executions, one of them accesses the unobserved block of size `10` via the guessed address `0x100` at line 4 while the others do not reach line 4 thereby raising no UB. Therefore, with this simpler semantics the compiler would (annoyingly) have to conclude that line 4 can access object `a`.

4.12 Sometimes Two Blocks Are Not Enough

Justifying the correctness of some optimizations requires more than two blocks. We give an example of an optimization that requires triple allocation ($N = 2$ in our semantics). This optimization removes single observations of addresses of local variables through comparison with a function argument. For example, in the following function on the left, if the address of `c` is unobserved except at line 3, the comparison can be folded to `false` to get the optimized code on the right. This enables further optimizations since the address of `c` becomes completely unobserved in the target code and thus the compiler can assume that the block `c` cannot be accessed via a guessed address.

```
1: void foo(int i) {                          void foo(int i) {
2:   char c[4];                                 char c[4];
3:   if (c == (char*)i) {                       if (false) {
4:     ...                                        ...
5:   } else {                        ⇒          } else {
6:     *(char*)0x200 = 0;                         *(char*)0x200 = 0;
7:   }                                          }
8:   ...                                        ...
9: }                                          }
```

The reason why two blocks are not enough for this optimization is that `c` can be accessed in line 6 of the original code when memory has only space for two blocks: The observation of the address of `c` in line 3 can forbid one of the twin executions from reaching line 6, which does not happen in the optimized code. To see this clearly, suppose that `i = 0x100` and the twin blocks for `c` can be only allocated at `0x100` and `0x200` because the memory had space for just those blocks. Then, in the original code, none of the twin executions triggers UB for the guessed access at line 6, while in the optimized code, the execution with `c = 0x100` triggers UB at line 6 now that guessing is prevented, as usual, by the twin allocations.

However, if we have triple twin allocations, `c` cannot be accessed via a guessed address even if one of the triple twin executions is ruled out by a single observation: We still have two twin executions left. For example, in the above setting, if `c` is allocated at `0x100`, `0x200` and `0x300`, in the original code the execution with `c = 0x300` triggers UB for the guessed access at line 6.

We can argue as follows that triple twin allocations enable the optimization above for a fresh block `c` when the address of `c` is observed only once in the comparison. If one of the triple executions in the optimized code accesses `c` via a guessed address (causing UB in the optimized code), we can always trigger UB for the guessed access in one of the three possible executions of the original code as discussed above. If not, the triple executions behave exactly the same in the optimized code because there is no observation of the address of `c`, and the behavior is simulated by one of the triple executions of the original code that makes the comparison `c == p` false.

Finally, we note that over-approximating the number of twin blocks is sound. However, as we have seen in the previous example, under-approximating it is not. In practice, due to the limited reasoning power of compilers in general, and LLVM in particular, we believe 3 blocks are sufficient. [5]

[5] We are only aware of one optimization in LLVM and GCC that may require more than three blocks, and further investigation is ongoing to determine whether it is valid (see http://llvm.org/PR35102).